

Arquitectura — Orion

Productora de eventos (Chile) — sitio web + gestión de contenido

Rama migracion-django-vue · Stack de 3 servicios (Django REST + Express BFF + Vue)

RESUMEN

El proyecto está migrando de una SPA React que hablaba directo con Supabase (Postgres + RLS) hacia una arquitectura de **3 servicios independientes**: un frontend Vue estático, un BFF (backend-for-frontend) en Express que actúa como proxy/gateway, y una API Django REST Framework que es la única dueña del esquema sobre la misma base Postgres de Supabase (ya sin RLS ni supabase-js). El dominio de datos sigue en español (tablas, campos, slugs).

DIAGRAMA DE ARQUITECTURA



El BFF es el único origen público que el SPA conoce — Django nunca es llamado directamente desde el navegador.

COMPONENTES

frontend-vue

- Vue 3 (Composition API) + TypeScript + Vite + Bootstrap 5
- 6 vistas: Landing, Servicios, Portafolio(+Detalle), Blog(+Detalle)
- `services/*.ts` → instancia axios compartida (`api.ts`), solo rutas relativas
- Config: `VITE_API_URL` → URL del BFF
- Build estático → publicado en GitHub Pages

bff-express

- Express 4 + TypeScript, un solo archivo (`src/index.ts`, ~38 líneas)
- **Proxy genérico**: todo `/api/*` se reenvía tal cual a Django (método, query, body) vía axios
- Sin lógica propia, sin auth, sin caché, sin agregación de llamadas
- Resuelve CORS: el navegador solo ve este origen
- Config: `DJANGO_API_URL`, `PORT`, `ALLOWED_ORIGINS`

backend-django

- Django 5 + Django REST Framework, un solo app: `catalogo`
- Único dueño del esquema (ORM/migraciones) sobre Postgres
- Conexión vía `django-environ` + `django-database-url` (`DATABASE_URL`, pooler Supabase)
- Permisos DRF: `AllowAny` — sin login, filtra contenido publicado en cada endpoint
- Deploy prod: `gunicorn orion_api.wsgi`

Supabase Postgres

- Usado **solo como base de datos** — sin `supabase-js`, sin RLS activo
- Django se conecta como rol privilegiado ⇒ RLS ya no protege el acceso
- El control de qué se expone queda en los endpoints (filtros activo/publicado/estado)
- Datos existentes preservados vía `migrate --fake-initial`

MODELO DE DATOS (APP CATALOGO)

- `CategoriaServicio` — nombre, slug, orden
- `Servicio` — pertenece a categoría, activo/orden
- `EventoTipo` — nombre, slug
- `Evento` — tipo, cliente, fecha, publicado/destacado
- `FotoEvento` — galería de un evento
- `Tag / Post / PostTag` — blog con tags (M2M)
- `Cotizacion` — único modelo con escritura pública (formulario de contacto)

FLUJO DE UNA REQUEST

- 1 **GET /servicios/** — el navegador pide datos a `frontend-vue`, que llama a `VITE_API_URL` (el BFF) con la ruta relativa `/servicios/`.
- 2 **bff-express** recibe en `/api/servicios/`, aplica CORS y reenvía la misma request (método/query/body) a `DJANGO_API_URL + /api/servicios/` vía `axios`.
- 3 **backend-django** resuelve la vista DRF (`ServicioList`), filtra `activo=True`, consulta Postgres vía el ORM y serializa la respuesta.
- 4 La respuesta sube sin transformar por el mismo camino: Django → BFF (status/body tal cual) → Vue, que la renderiza.

POST /cotizaciones/ sigue el mismo camino de punta a punta (SPA → BFF → Django → Postgres); no hay envío de email de notificación en ningún punto del stack nuevo — la cotización solo queda persistida en la tabla.

SEGURIDAD Y CONTROL DE ACCESO

- Sin autenticación de usuarios: DRF corre con `DEFAULT_PERMISSION_CLASSES: AllowAny` en toda la API.
- `django.contrib.auth` está instalado solo porque DRF lo necesita internamente (`request.user` → `AnonymousUser`), no habilita login.
- RLS de Supabase ya no aplica (Django conecta con credenciales privilegiadas) — la visibilidad de contenido no publicado depende 100% de los filtros en cada vista (`activo, publicado, estado='publicado'`).
- CORS se controla en dos capas: `CORS_ALLOWED_ORIGINS` (Django) y `ALLOWED_ORIGINS` (BFF) — ambas deben incluir el dominio de GitHub Pages en producción.
- `DATABASE_URL` (con password) solo vive en `.env` (gitignored) o en el panel del host — nunca en el frontend.

ENTORNOS Y DESPLIEGUE

SERVICIO	TECNOLOGÍA	HOSTING	PUERTO LOCAL
frontend-vue	Vue 3 + Vite (build estático)	GitHub Pages	5173
bff-express	Express + TS	Render / Railway	3001
backend-django	Django + DRF + gunicorn	Render / Railway	8000
Postgres	—	Supabase (solo DB)	—

Actualmente no hay CI/CD para los 3 servicios nuevos (no hay Dockerfile, docker-compose ni workflow de GitHub Actions que los construya); se levantan y despliegan de forma independiente, 3 terminales en local.

PIEZAS HEREDADAS (PRESENTES EN EL REPO, NO EN USO)

La raíz del repo todavía contiene la app original React + Tailwind + Framer Motion que hablaba directo con Supabase vía supabase-js (carpeta `src/`, `index.html`, `tailwind.config.js`), un Express intermedio en `server/` (rutas `servicios/portafolio/blog/cotizaciones` contra Supabase directo), y un stub vacío en `supabase-node-api/`. El único workflow de GitHub Actions (`.github/workflows/deploy.yml`) y el `netlify.toml` raíz todavía apuntan a esa app vieja, no al stack nuevo. Según el `README.md`, la app React original se mantiene como referencia en la rama `main`.